Guía de instalación y puesta en marcha

Plataforma de Clubs de Lectura — TFGdaw

Índice

1. Requisitos previos
2. Obtener el código fuente
3. Configurar las variables de entorno
4. Construir y arrancar los contenedores
5. Ejecutar las migraciones de base de datos
6. Verificar que todo funciona
7. Apagar el proyecto
8. Operaciones habituales
9. Solución de problemas frecuentes

---

1. Requisitos previos

Antes de empezar, asegúrate de tener instalado en tu máquina:

Herramienta	Versión mínima	Cómo verificarlo
Docker Desktop (Windows / macOS) o **Docker Engine** (Linux)	24.x	`docker --version`
Docker Compose	2.x (incluido en Docker Desktop)	`docker compose version`
Git	cualquiera	`git --version`

> ***No necesitas*** *tener instalados PHP, Node.js, Composer ni ningún otro lenguaje o runtime. Todo corre dentro de los contenedores Docker.*

Requisitos de hardware recomendados

- 4 GB de RAM disponibles para Docker
- 3 GB de espacio en disco (imágenes + volúmenes de datos)
- Conexión a Internet para descargar las imágenes y las dependencias en el primer arranque

---

2. Obtener el código fuente

Clona el repositorio en tu máquina:

```
git clone <url-del-repositorio>
cd TFGdaw
```

La estructura que encontrarás es la siguiente:

```
TFGdaw/
├── backend/          # API REST en Symfony 7.4
```

```
├── frontend/         # SPA en React 18 + TypeScript
├── docker/
│   └── nginx/
│       └── default.conf   # Configuración del reverse proxy
├── docker-compose.yml
└── .env.example      # Plantilla de variables de entorno
```

---

3. Configurar las variables de entorno

Copia la plantilla de variables de entorno:

```
# Linux / macOS
cp .env.example .env

# Windows (PowerShell)
Copy-Item .env.example .env
```

Abre el archivo `.env` con cualquier editor de texto y revisa los valores:

```
# ── Base de datos ──────────────────────────────────────────────────────
DB_ROOT_PASSWORD=rootpassword      # Contraseña del usuario root de MariaDB
DB_NAME=tfgdaw                     # Nombre de la base de datos
DB_USER=tfgdaw                     # Usuario de la aplicación
DB_PASSWORD=tfgdaw_secret          # Contraseña del usuario de la aplicación

# ── Symfony ────────────────────────────────────────────────────────────
APP_SECRET=cambia_esto_por_una_clave_aleatoria_de_32_chars

# ── Google Books API (opcional) ────────────────────────────────────────
GOOGLE_BOOKS_API_KEY=

# ── Puerto HTTP ────────────────────────────────────────────────────────
HTTP_PORT=80
```

Valores que debes cambiar

`APP_SECRET` — Es obligatorio cambiarlo. Genera una clave segura con uno de estos comandos:

```
# Linux / macOS
openssl rand -hex 32

# Windows (PowerShell)
[System.Web.Security.Membership]::GeneratePassword(32, 0)

# Alternativa online: https://generate-secret.vercel.app/32
```

Copia el resultado y ponlo como valor de `APP_SECRET`.

`GOOGLE_BOOKS_API_KEY` — Es opcional. La búsqueda de libros funciona sin clave pero Google aplica un límite de peticiones más restrictivo. Si vas a usarlo en producción o de forma intensiva, obtén una clave gratuita en [Google

Cloud Console activando la API "Books API".

`HTTP_PORT` — Cámbialo si el puerto 80 ya está en uso en tu máquina (por ejemplo, ponlo a `8080`).

> *Los valores de las contraseñas de base de datos (`DB_ROOT_PASSWORD`, `DB_PASSWORD`) pueden dejarse como están para un entorno de desarrollo local. En producción, usa contraseñas seguras.*

---

4. Construir y arrancar los contenedores

Desde la carpeta raíz del proyecto (`TFGdaw/`), ejecuta:

```
docker compose up --build -d
```

Este único comando hace todo lo siguiente de forma automática:

1. **Descarga** las imágenes base de Docker Hub (MariaDB, Nginx, PHP, Node).
2. **Construye** la imagen del backend: instala extensiones PHP, Composer, dependencias de producción y genera la caché de Symfony.
3. **Construye** la imagen del frontend: compila el proyecto React con TypeScript y Vite, y empaqueta el resultado en una imagen Nginx ligera.
4. **Arranca** los cuatro servicios: base de datos, PHP-FPM, frontend y reverse proxy Nginx.

La primera vez puede tardar **entre 3 y 8 minutos** dependiendo de tu conexión y tu máquina. Las siguientes veces será mucho más rápido porque Docker reutiliza las capas en caché.

Arquitectura de los contenedores

```
Tu navegador
     │
     ▼ :80
 ┌─────────┐
 │  nginx  │  ← reverse proxy (imagen: nginx:1.27-alpine)
 └─────────┘
      │
      ├─ /api/*      → ┌──────────┐       ┌──────────┐
      │                │   php    │ ──────▶│    db    │
      │                │ (FPM)    │       │ MariaDB  │
      │                └──────────┘       └──────────┘
      │
      ├─ /uploads/*  → volumen del disco (avatares, imágenes)
      │
      └─ /*          → ┌──────────┐
                       │ frontend │
                       │ (React SPA) │
                       └──────────┘
```

Comprobar que los contenedores están en marcha

```
docker compose ps
```

Deberías ver los cuatro servicios con estado `running` (o `healthy` para la base de datos):

```
NAME                STATUS
tfgdaw-db-1         running (healthy)
tfgdaw-php-1        running
tfgdaw-frontend-1   running
tfgdaw-nginx-1      running
```

---

5. Ejecutar las migraciones de base de datos

Este paso es **obligatorio** la primera vez y debe ejecutarse cuando la base de datos ya está en marcha (estado `healthy`):

```
docker compose run --rm migrate
```

Este comando levanta un contenedor temporal que aplica todas las migraciones de Doctrine para crear las tablas de la base de datos, y luego se elimina solo.

Verás una salida similar a esta:

```
[notice] Migrating up to DoctrineMigrations\Version20250101000000
[notice] finished in 1.2s, used 20M memory, 12 migrations executed, 12 sql queries
```

> *Si ves el mensaje `No migrations to execute`, significa que la base de datos ya estaba actualizada. Es un resultado correcto.*

---

6. Verificar que todo funciona

Abre tu navegador y ve a:

```
http://localhost
```

(O `http://localhost:8080` si cambiaste el `HTTP_PORT`.)

Deberías ver la página de inicio de la plataforma con el hero, los botones de registro y la sección de funcionalidades.

Comprobaciones adicionales

Verificar la API:

```
http://localhost/api/auth/me
```

Debe responder con `{"error":"No autenticado"}` — esto confirma que Symfony está respondiendo correctamente.

Crear una cuenta de administrador (opcional):

Para acceder al panel de administración en `/admin`, la primera cuenta que crees necesita el rol de administrador. Puedes asignárselo directamente en la base de datos:

```
docker compose exec db mariadb -u tfgdaw -ptfgdaw_secret tfgdaw \
  -e "UPDATE user SET is_admin = 1 WHERE email = 'tu@email.com';"
```

O bien regístrate normalmente y luego otra cuenta admin puede promoverte desde `/admin → Usuarios`.

---

7. Apagar el proyecto

Para detener los contenedores sin borrar los datos:

```
docker compose down
```

Para detener y **eliminar también todos los datos** (base de datos, uploads):

```
docker compose down -v
```

> *Usa `-v` con precaución: borra el volumen de la base de datos y todos los archivos subidos (avatares e imágenes de posts). Es irreversible.*

---

8. Operaciones habituales

Volver a arrancar tras apagar

```
docker compose up -d
```

No hace falta `--build` a menos que hayas modificado el código fuente.

Reconstruir tras cambios en el código

```
docker compose up --build -d
```

Ver los logs en tiempo real

```
# Todos los servicios
docker compose logs -f

# Solo el backend (Symfony)
docker compose logs -f php

# Solo el reverse proxy
docker compose logs -f nginx
```

Acceder a la consola de Symfony

```
docker compose exec php php bin/console <comando>

# Ejemplos:
docker compose exec php php bin/console doctrine:migrations:status
```

```
docker compose exec php php bin/console cache:clear
docker compose exec php php bin/console debug:router
```

Acceder a la base de datos

```
docker compose exec db mariadb -u tfgdaw -ptfgdaw_secret tfgdaw
```

Crear una nueva migración tras modificar entidades

```
docker compose exec php php bin/console doctrine:migrations:diff
docker compose run --rm migrate
```

---

9. Solución de problemas frecuentes

El puerto 80 ya está en uso

```
Error: Bind for 0.0.0.0:80 failed: port is already allocated
```

Solución: Edita `.env` y cambia `HTTP_PORT=80` por otro puerto (p. ej. `HTTP_PORT=8080`), luego vuelve a ejecutar `docker compose up -d`.

La base de datos tarda en arrancar y el servicio `php` falla

El servicio `php` espera a que `db` pase el healthcheck antes de arrancar. Si el primer arranque tarda mucho (más de 60 segundos), puede ocurrir un timeout. Simplemente vuelve a ejecutar:

```
docker compose up -d
```

Docker reiniciará solo los servicios que fallaron.

El comando `migrate` falla con "Access denied"

Asegúrate de que los valores de `DB_USER`, `DB_PASSWORD` y `DB_NAME` en `.env` coinciden exactamente con los que arrancaron la base de datos. Si cambiaste las credenciales después del primer arranque, la base de datos ya tenía el usuario anterior creado. Solución: elimina el volumen y recrea la base de datos:

```
docker compose down -v
docker compose up --build -d
docker compose run --rm migrate
```

La búsqueda de libros no devuelve resultados

Google Books API tiene un límite de peticiones por IP cuando no se usa clave de API. Si ves errores 429, añade tu `GOOGLE_BOOKS_API_KEY` en `.env` y reconstruye:

```
docker compose up -d php
```

(Solo es necesario reiniciar el servicio `php`, no reconstruir todo.)

Las imágenes subidas no se ven

Los avatares y las fotos de posts se guardan en el volumen `uploads`. Verifica que el volumen está montado correctamente:

```
docker compose exec nginx ls /var/www/html/public/uploads/
```

Si el directorio está vacío es normal si aún no se ha subido ningún archivo.

Cómo resetear completamente el proyecto

Si quieres volver al estado inicial (sin datos, sin caché):

```
docker compose down -v          # Elimina contenedores y volúmenes
docker compose up --build -d    # Reconstruye y arranca
docker compose run --rm migrate # Aplica migraciones
```

Guía generada para el Trabajo de Fin de Grado — DAW